

Design Pattern Mapping - Detailed

For every pattern: **Where used** → **Problem solved** → **Structure** → **Code sketch** → **Flow in the flash sale** → **Alternatives rejected** → **Trade-offs** → **How we test it**.

0. Pattern Catalogue (overview)

#	Pattern	Category	Module(s)	Primary problem solved
1	Strategy	Behavioural	Pricing, purchase limit, delivery selection	Interchangeable rules without conditionals
2	Factory	Creational	PaymentProviderFactory	Create the right provider adapter from configuration
3	State	Behavioural	Reservation, Order, Payment lifecycle	Valid transitions enforced in one place
4	Observer	Behavioural	Order events → Notification, Shipment, Analytics	Decoupled side effects
5	Adapter	Structural	Payment gateways, couriers, SMS/email	Uniform interface over third-party APIs
6	Repository	Architectural	All persistence	Hide SQL; test with fakes; keep atomic operations explicit
7	Facade	Structural	CheckoutFacade	One simple entry for a multi-step workflow
8	Circuit Breaker	Resilience	Gateway, courier, notification calls	Stop cascading failure
9	Decorator	Structural	ResilientProviderDecorator, metrics/logging wrappers	Add retry/timeout/metrics without changing providers
10	Saga (orchestration)	Distributed	CheckoutSaga	Distributed transaction without 2PC
11	Transactional Outbox	Messaging	Every event-publishing service	Atomic state change + event publication
12	Idempotent Receiver	Messaging	Consumers, IdempotencyGuard	Safe redelivery and retries

1. Strategy

Where: PricingStrategy (FlashSalePricing, CouponPricing, BulkDiscountPricing), PurchaseLimitPolicy, DeliveryPartnerSelector. **Problem:** Price rules differ per sale and change often; if/else chains in checkout are fragile. **Structure:** CheckoutFacade → PricingStrategy ← FlashSalePricing | CouponPricing | ...

```
interface PricingStrategy { Money price(Cart cart, Customer c, Sale sale); }
class FlashSalePricing implements PricingStrategy {
    public Money price(Cart cart, Customer c, Sale s) { return s.priceFor(cart.productId()).times(cart.qty()); }
}
class CompositePricing implements PricingStrategy { // chain: base price then coupon
    List<PricingStrategy> steps; ...
}
```

Flow: the sale configuration names the strategy list (FLASH, COUPON); PricingEngine builds the chain at start-up; price is re-computed **server-side** at checkout (never trusting client price). **Alternatives rejected:** hard-coded conditionals (not extensible); rules engine (Drools) – too heavy for the case. **Trade-offs:** more classes; ordering of composed strategies must be explicit (discount order changes the total). **Test:** table-driven tests per strategy + a golden test for the combined chain.

2. Factory

Where: PaymentProviderFactory.get(code), NotificationChannelFactory. **Problem:** Provider selection (by customer choice, region, health, or failover) should not be scattered.

```
class PaymentProviderFactory {
    private final Map<String, Supplier<PaymentProvider>> registry = new HashMap<>();
    void register(String code, Supplier<PaymentProvider> s) { registry.put(code, s); }
    PaymentProvider get(String code) {
        var p = Optional.ofNullable(registry.get(code)).orElseThrow(UnknownProviderException::new);
        return new ResilientProviderDecorator(p.get(), breakers.forProvider(code), retryPolicy);
    }
}
```

Flow: PaymentService.initiate calls factory.get(order.preferredProvider()); if the breaker for the primary provider is OPEN, the factory returns the configured secondary (failover policy). **Alternatives:** service locator / new inside services (hidden dependency, hard to test). **Trade-off:** registry must match configuration; misconfiguration found only at start-up → mitigated by a start-up validation step that fails fast. **Test:** unknown code → exception; failover returns secondary when primary breaker is open.

3. State

Where: ReservationState (Reserved, PaymentPending, Confirmed, Released, Sold), OrderState, PaymentState. **Problem:** Many if (state == ...) checks scattered; illegal transitions (e.g. SOLD → RELEASED) cause stock corruption.

```

interface ReservationState {
    default ReservationState confirm() { throw new IllegalStateTransitionException(this, "confirm"); }
    default ReservationState release(Reason r) { throw new IllegalStateTransitionException(this, "release"); }
    default ReservationState expire() { throw new IllegalStateTransitionException(this, "expire"); }
}
class PaymentPendingState implements ReservationState {
    public ReservationState confirm() { return new ConfirmedState(); }
    public ReservationState release(Reason r) { return new ReleasedState(r); }
    public ReservationState expire() { return new ReleasedState(Reason.TIMEOUT); }
}

```

Database mirror (important): the in-memory State object is never trusted alone. Every transition is persisted with a guarded update: `UPDATE inventory_reservation SET state='RELEASED' WHERE reservation_id=? AND state IN ('RESERVED','PAYMENT_PENDING')` – if 0 rows, another actor already moved it (idempotent no-op). **Transition table (reservation):** | From | Event | To | |-----|-----|----| | RESERVED | payment initiated | PAYMENT_PENDING | | RESERVED / PAYMENT_PENDING | timeout, cancel, payment failed | RELEASED | | PAYMENT_PENDING | payment succeeded | CONFIRMED | | CONFIRMED | order confirmed | SOLD | | CONFIRMED | order creation impossible + refund | RELEASED | **Alternatives:** enum + switch (grows unmanageably); workflow engine (heavy). **Trade-off:** one class per state; duplication between code state machine and DB constraint (we keep both: code for clarity, DB for safety). **Test:** exhaustive matrix test (every state x every event) asserting allowed/forbidden.

4. Observer

Where: `OrderEventObserver` implementations (`NotificationObserver`, `ShipmentObserver`, `AnalyticsObserver`); across services realised as Kafka subscriptions. **Problem:** `OrderService` must not know who cares about order changes.

```

interface OrderEventObserver { void onEvent(OrderEvent e); }
class OrderEventDispatcher { List<OrderEventObserver> observers; void publish(OrderEvent e){ observers.forEach(o -> safely(o, e)); } }

```

Flow: `OrderConfirmed` → Shipment creates shipment; Notification sends email/SMS; Inventory marks SOLD; each consumer failure is isolated (retry/DLQ) and never rolls back the order. **Alternatives:** direct calls from `OrderService` (tight coupling, failure propagation). **Trade-offs:** implicit flow is harder to follow → tracing + event catalogue; ordering only per partition key. **Test:** failing observer does not affect others or the order state.

5. Adapter

Where: `RazorpayAdapter`, `StripeAdapter`, `DelhiveryAdapter`, `TwilioSmsAdapter`. **Problem:** Each vendor has different request shapes, error codes, auth and webhook formats. **Responsibilities of an adapter:** map our `ChargeRequest` to the vendor payload, attach `merchantRef` / idempotency header, translate vendor status/error codes to `PENDING/SUCCESS/DECLINED` or `Transient/PermanentException`, verify webhook signatures, hide pagination/quirks. **Alternatives:** calling vendor SDKs directly (vendor lock-in in domain code). **Trade-off:** lowest-common-denominator features; vendor-specific data kept in an `extras` map. **Test:** shared contract tests (see LSP) + recorded vendor responses (stubs).

6. Repository

Where: `InventoryRepository`, `ReservationRepository`, `PaymentRepository`, `OrderRepository`. **Problem:** SQL scattered in services; hard to test; atomic semantics must stay explicit.

```

interface InventoryRepository {
    boolean tryReserve(ProductId p, int qty); // single atomic UPDATE ... WHERE available_quantity >= qty
    void releaseUnits(ProductId p, int qty);
    void markSold(ProductId p, int qty);
}

```

Important: `tryReserve` is deliberately **not** `find + save`; a read-modify-write would race. The repository exposes the atomic operation as its contract. **Alternatives:** active record / ORM entity with `@Version` (retry storm under contention for the hot row). **Trade-off:** the abstraction exposes DB-aware semantics (atomic ops); documented in the interface contract. **Test:** integration test with real PostgreSQL (Testcontainers) running many concurrent `tryReserve` calls; assert exactly N successes.

7. Facade

Where: `CheckoutFacade.checkout(CheckoutRequest)`. **Problem:** Clients (web, mobile) should not orchestrate pricing, reservation validation, payment creation and error mapping. **Flow:** validate request → load reservation → price via strategy → start saga → return `202 {paymentId, redirectUrl}`. **Guard against god class:** the Facade holds **no business rules**; it delegates to `PricingEngine`, `CheckoutSaga`, `PaymentClient`. **Trade-off:** a single entry point can become a bottleneck or a dumping ground; reviewed via a "no logic in facade" rule.

8. Circuit Breaker

Where: around payment gateway, courier APIs, SMS/email providers. **States:** CLOSED → (failure rate ≥ 50% over 20 calls in 30 s, or 10 consecutive timeouts) → OPEN → (after 20 s) → HALF_OPEN (3 probe calls) → CLOSED or OPEN. **Behaviour when OPEN:** fail fast (no thread held), payments set to `PENDING_RETRY`, customer shown "payment processing", failover provider chosen if configured, reservation TTL grace extension once. **Alternatives:** plain retries (retry storms deepen outages); bulkheads alone (do not stop calls). **Trade-offs:** tuning required; false OPEN rejects good traffic → metrics `breaker_state` and alert. **Test:** chaos test with `ChaosProvider` injecting 60% failures; assert breaker opens and calls are short-circuited.

9. Decorator

Where: `ResilientProviderDecorator`, `MeteredProviderDecorator`, `LoggingProviderDecorator`.

```

class ResilientProviderDecorator implements PaymentProvider {
    ChargeResponse charge(ChargeRequest r) {
        return circuitBreaker.executeSupplier(() ->
            retry.executeSupplier(() ->
                timeLimiter.executeSupplier(() -> delegate.charge(r)));
    }
}

```

Order matters: timeout innermost, then retry, then breaker outermost (so repeated retries are counted by the breaker). **Alternatives:** AOP interceptors (less explicit, harder to reason about order). **Trade-off:** nested wrappers can complicate stack traces; trace spans label each layer.

10. Saga (orchestration)

Where: CheckoutSaga in Checkout Service. **Steps and compensations:** | Step | Action | Compensation | |-----|-----|-----| | 1 | Validate reservation (RESERVED, not expired) | - | | 2 | Create payment (idempotent) | cancel payment intent | | 3 | Await payment result (event/webhook) | - | | 4 | Confirm reservation | release reservation | | 5 | Create order (via event) | refund + release (if impossible) | Saga state is persisted (SAGA_STATE : step, status, deadline), so a crashed orchestrator resumes; deadlines trigger reconciliation. **Alternatives:** 2PC/XA (locks, poor availability); pure choreography (hard to see the whole workflow, harder compensation). **Trade-offs:** eventual consistency; compensation logic must itself be idempotent and retryable.

11. Transactional Outbox

Where: Inventory, Payment, Order, Shipment. **Problem:** "update DB then publish to Kafka" can lose or duplicate events if the process crashes between them. **Mechanism:** in **one local transaction** write the state change and an outbox_event row; a relay (poller or CDC such as Debezium) publishes unpublished rows and marks published_at. **Guarantee:** at-least-once publication, ordered per aggregate. Duplicates are expected → consumers are idempotent (pattern 12). **Alternatives:** dual write (inconsistent), publish-then-commit (phantom events). **Trade-offs:** extra table and relay; slight publication delay (target < 1 s; alert > 10 s).

12. Idempotent Receiver

Where: every consumer and every mutating API. **Mechanisms:** | Boundary | Mechanism | |-----|-----| | Reservation API | UNIQUE(customer_id, idempotency_key) | | Payment API | UNIQUE(idempotency_key), UNIQUE(reservation_id), merchant_ref to gateway | | Order creation | UNIQUE(reservation_id) + processed_event(consumer, event_id) | | Notification | UNIQUE(dedupe_key) | | Webhook | gateway_event(gateway_event_id) primary key | **Rule:** same key + same payload → same response; same key + different payload → 422. **Trade-off:** storage and cleanup of dedupe records (TTL 24 h for API keys, 7 days for event ids).

13. Pattern Interaction in the Purchase Flow

```

Client → CheckoutFacade (Facade)
    → PricingEngine → PricingStrategy (Strategy)
    → CheckoutSaga (Saga) → ReservationService (Repository + State)
        → PaymentService → PaymentProviderFactory (Factory)
            → ResilientProviderDecorator (Decorator + Circuit Breaker)
                → RazorpayAdapter (Adapter)
Payment webhook → Outbox → Kafka → Idempotent Receiver
OrderService (State) → OrderEvent (Observer) → Notification / Shipment / Inventory

```

14. Patterns Deliberately NOT Used

Pattern	Reason
Singleton for services	Hidden global state; DI container manages lifetimes
Distributed lock (Redis RedLock) for stock	Weaker guarantees than a DB atomic update; adds failure modes
Event Sourcing for inventory	Powerful but operationally heavy; audit trail from history tables is sufficient
2PC / XA	Locks and poor availability across services

15. Jury Q&A

Q: Which pattern is most important for correctness? State + Repository (guarded atomic updates) for stock; Idempotent Receiver + Outbox for payment/order. **Q: Which for resilience?** Circuit Breaker + Decorator + Saga + Outbox. **Q: What does each pattern cost?** See "Trade-offs" in every section; the biggest costs are traceability of event flows and the duplication of state rules between code and DB (accepted for safety).